

AMSC460: Homework 3

JAMES ZHANG*

September 15, 2024

Solution.

- (a) Denote $f = x + \ln x = 0$. f is continuous and differentiable on the interval on $(0, \infty)$. Note that $f(0.1) \approx -2.2$ and $f(1) = 1$. Since $0 \in (-2.2, 1)$, by the Intermediate Value Theorem, f attains all values in between, so $\exists x^* \in [0.1, 1]$ such that $f(x^*) = 0$. To show that there is *exactly* one root, assume not and that there are 2 solutions such that

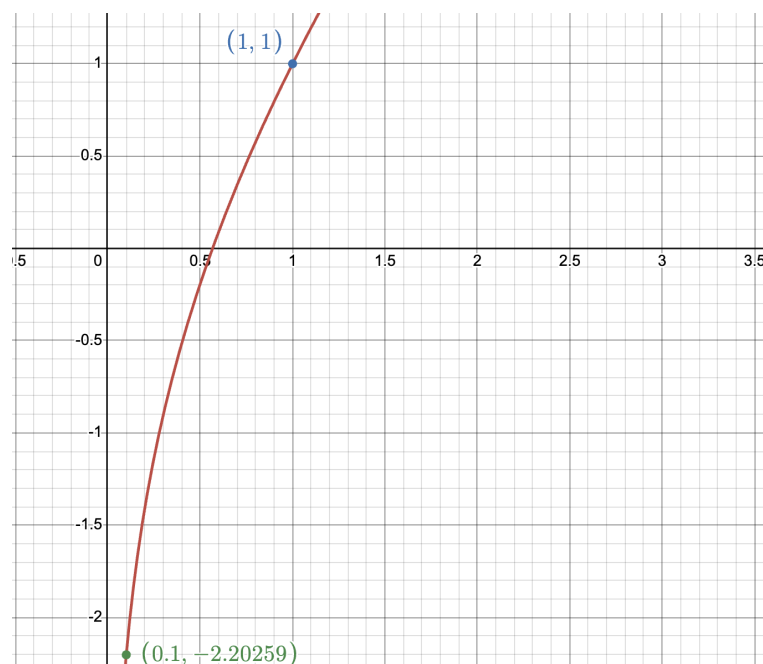
$$f(a) = 0 = f(b)$$

By Rolle's Theorem, $\exists c \in (a, b)$ such that $f'(c) = 0$. However, $f'(x) = 1 + \frac{1}{x} = 0$

$$f'(c) = 1 + \frac{1}{c} = 0 \implies \frac{1}{c} = -1 \implies c = -1$$

which is a contradiction because $c \in [0.1, 1]$ and so therefore, there must be *exactly* one root.

- (b) See the below graph, plotted using Desmos' Graphing Calculator



*Email: jzhang72@terpmail.umd.edu

- (c) Let us apply the mentioned methods. Here is the driver code I use throughout this process.

```

1 function find_roots()
2     f = @(x) x + log(x);
3     df = @(x) 1 + 1/x;
4     % Convergence criterion
5     tol = 1e-10;
6     max_iter = 100;
7     % 1. Bisection Method
8     [x_bisect, iter_bisect] = bisection(f, 0.5, 0.6, tol);
9     % 2. Fixed Point Iteration
10    g = @(x) exp(-x);
11    [x_fixed, iter_fixed] = fixed_point(g, 0.5, tol, max_iter);
12    % 3. Newton's Method
13    [x_newton, iter_newton] = newton(f, df, 0.5, tol, max_iter);
14    % 4. Secant Method
15    [x_secant, iter_secant] = secant(f, 0.5, 0.6, tol, max_iter);
16    fprintf('Method      | Root          | Iterations\n');
17    fprintf('Bisection | %.10f | %d\n', x_bisect, iter_bisect);
18    fprintf('Fixed Point | %.10f | %d\n', x_fixed, iter_fixed);
19    fprintf('Newton     | %.10f | %d\n', x_newton, iter_newton);
20    fprintf('Secant     | %.10f | %d\n', x_secant, iter_secant);
21 end

```

- i. For the Bisection Method, the interval $[0.5, 0.6]$ is valid because $f(0.5) * f(0.6) < 0$, and so since the endpoints are opposite signs, by the IVT, the function attains all values in between, and so there must be at least one root in the interval.

```

1 function [x, iter] = bisection(f, a, b, tol)
2     iter = 0;
3     while (b - a) / 2 > tol
4         p = (a + b) / 2;
5         if f(p) == 0
6             break;
7         elseif f(a) * f(p) < 0
8             b = p;
9         else
10            a = p;
11        end
12        iter = iter + 1;
13        fprintf('Bisection Method Iteration %d: x = %.10f\n', iter, p);
14    end
15    x = (a + b) / 2;
16    fprintf('Error: %.10f\n', abs(p - fzero(f, [0.5, 0.6])))
17 end

```

```

Bisection Method Iteration 1: x = 0.5500000000
Bisection Method Iteration 2: x = 0.5750000000
Bisection Method Iteration 3: x = 0.5625000000
Bisection Method Iteration 4: x = 0.5687500000
Bisection Method Iteration 5: x = 0.5656250000
Bisection Method Iteration 6: x = 0.5671875000
Bisection Method Iteration 7: x = 0.5664062500
Bisection Method Iteration 8: x = 0.5667968750
Bisection Method Iteration 9: x = 0.5669921875
Bisection Method Iteration 10: x = 0.5670898437
Bisection Method Iteration 11: x = 0.5671386719
Bisection Method Iteration 12: x = 0.5671630859
Bisection Method Iteration 13: x = 0.5671508789
Bisection Method Iteration 14: x = 0.5671447754
Bisection Method Iteration 15: x = 0.5671417236
Bisection Method Iteration 16: x = 0.5671432495
Bisection Method Iteration 17: x = 0.5671440125
Bisection Method Iteration 18: x = 0.5671436310
Bisection Method Iteration 19: x = 0.5671434402
Bisection Method Iteration 20: x = 0.5671433449
Bisection Method Iteration 21: x = 0.5671432972
Bisection Method Iteration 22: x = 0.5671432734
Bisection Method Iteration 23: x = 0.5671432853
Bisection Method Iteration 24: x = 0.5671432912
Bisection Method Iteration 25: x = 0.5671432883
Bisection Method Iteration 26: x = 0.5671432897
Bisection Method Iteration 27: x = 0.5671432905
Bisection Method Iteration 28: x = 0.5671432901
Bisection Method Iteration 29: x = 0.5671432903
Error: 0.0000000001
Method      | Root      | Iterations
-----|-----|-----
Bisection   | 0.5671432904 | 29

```

The expected number of iterations using Bisection Method is

$$\text{ceil}\left(\frac{\log(b-a) - \log(2 * \text{atol})}{\log 2}\right) = \text{ceil}(28.89) = 29$$

and so our results match theoretical expectations.

- ii. For Fixed Point Iteration, define $f(x) = x + \ln x$. We want to cleverly choose $g(x)$ so it's easy to find x^* such that $f(x^*) = 0$. Consider $g(x) = -\exp(x)$

$$\begin{aligned}
 g(x) = \exp(-x) &\implies x^* = \exp(-x^*) \implies \ln(x^*) = -x^* \\
 &\implies x^* + \ln(x^*) = 0 \implies f(x^*) = 0
 \end{aligned}$$

```

1 function [x, iter] = fixed_point(g, x0, tol, max_iter)
2     x = x0;
3     iter = 0;
4     fprintf('Fixed Point Iteration:\n');
5     fprintf('Iteration 0: x = %.10f\n', x);
6     while iter < max_iter
7         x_new = g(x);
8         if abs(x_new - x) < tol
9             break;
10        end
11        x = x_new;

```

```

12     iter = iter + 1;
13     fprintf('Iteration %d: x = %.10f\n', iter, x);
14 end
15 fprintf('Error: %.10f\n', abs(x - fzero(@(x) x - exp(-x), [0.5,
    0.6])))
16 end

```

```

Fixed Point Iteration:
Iteration 0: x = 0.5000000000
Iteration 1: x = 0.6065306597
Iteration 2: x = 0.5452392119
Iteration 3: x = 0.5797030949
Iteration 4: x = 0.5600646279
Iteration 5: x = 0.5711721490
Iteration 6: x = 0.5648629470
Iteration 7: x = 0.5684380476
Iteration 8: x = 0.5664094527
Iteration 9: x = 0.5675596343
Iteration 10: x = 0.5669072129
Iteration 11: x = 0.5672771960
Iteration 12: x = 0.5670673519
Iteration 13: x = 0.5671863601
Iteration 14: x = 0.5671188643
Iteration 15: x = 0.5671571437
Iteration 16: x = 0.5671354337
Iteration 17: x = 0.5671477463
Iteration 18: x = 0.5671407633
Iteration 19: x = 0.5671447237
Iteration 20: x = 0.5671424776
Iteration 21: x = 0.5671437514
Iteration 22: x = 0.5671430290
Iteration 23: x = 0.5671434387
Iteration 24: x = 0.5671432063
Iteration 25: x = 0.5671433381
Iteration 26: x = 0.5671432634
Iteration 27: x = 0.5671433058
Iteration 28: x = 0.5671432817
Iteration 29: x = 0.5671432953
Iteration 30: x = 0.5671432876
Iteration 31: x = 0.5671432920
Iteration 32: x = 0.5671432895
Iteration 33: x = 0.5671432909
Iteration 34: x = 0.5671432901
Iteration 35: x = 0.5671432906
Iteration 36: x = 0.5671432903
Iteration 37: x = 0.5671432905
Error: 0.0000000001
Method      | Root          | Iterations
-----|-----|-----
Fixed Point | 0.5671432905 | 37

```

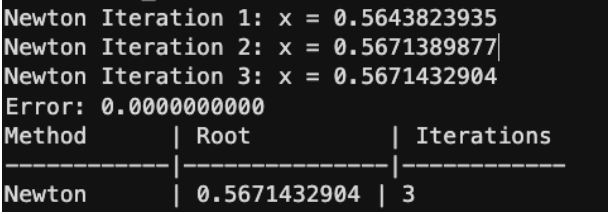
Note that $g(x) = \exp(-x) \implies g'(x) = -\exp(-x)$. Looking at the graph, $|g'(x)| < 0.65 \forall x \in [0.5, 0.6]$. Therefore, since $\rho < 1$, the uniqueness and convergence proofs which leverage First Order Taylor Series expansions provided in class are applicable, and our results match the theoretical expectations of linear convergence since the algorithm converges in 37 iterations.

iii. Newton's Method with $x_0 = 0.5$

```

1 function [x, iter] = newton(f, df, x0, tol, max_iter)
2   x = x0;
3   iter = 0;
4   while iter < max_iter
5       x_new = x - f(x) / df(x);
6       if abs(x_new - x) < tol
7           break;
8       end
9       x = x_new;
10      iter = iter + 1;
11      fprintf('Newton Iteration %d: x = %.10f\n', iter, x);
12  end
13  fprintf('Error: %.10f\n', abs(x - fzero(f, [0.5, 0.6])))
14 end

```



```

Newton Iteration 1: x = 0.5643823935
Newton Iteration 2: x = 0.5671389877
Newton Iteration 3: x = 0.5671432904
Error: 0.0000000000

```

Method	Root	Iterations
Newton	0.5671432904	3

Since we started in the neighborhood of the root and Newton's Method has quadratic convergence, we were able to converge to the root in only 3 iterations, which is much faster than the 37 iterations required for fixed point iteration, which is expected because fixed point iteration has linear convergence.

iv. Finally, the Secant Method with $x_0 = 0.5$ and $x_1 = 0.6$

```

1 function [x, iter] = secant(f, x0, x1, tol, max_iter)
2   iter = 0;
3   while iter < max_iter
4       x_new = x1 - f(x1) * (x1 - x0) / (f(x1) - f(x0));
5       if abs(x_new - x1) < tol
6           break;
7       end
8       x0 = x1;
9       x1 = x_new;
10      iter = iter + 1;
11      fprintf('Iteration %d: x = %.10f\n', iter, x1);
12  end
13  x = x1;
14  fprintf('Error: %.10f\n', abs(x - fzero(f, [0.5, 0.6])))
15 end

```

```
Iteration 1: x = 0.5684138975
Iteration 2: x = 0.5671202823
Iteration 3: x = 0.5671433068
Iteration 4: x = 0.5671432904
Error: 0.0000000000
Method      | Root      | Iterations
-----|-----|-----
Secant      | 0.5671432904 | 4
```

The Secant Method converges in 4 iterations. Recall that the Secant Method is guaranteed superlinear convergence, so this result makes sense that it's slower than Newton's Method which has quadratic convergence and faster than Fixed Point Iteration which has linear convergence.

□